

Шпаргалка Git — Урок 2

```

# --- НАЛАШТУВАННЯ (один раз) ---
git --version                                # перевірити встановлення
git config --global user.name "Твоє ім'я"    # автор комітів
git config --global user.email "mail@example.com"
git config --global init.defaultBranch main   # нові репо стартують з main
git config --list                             # переглянути налаштування

# --- СТВОРЕННЯ РЕПОЗИТОРІЮ ---
mkdir my_project && cd my_project
git init                                     # створити .git
git clone git@github.com:user/repo.git       # завантажити чужий/свій репо

# --- ЩОДЕННИЙ ЦИКЛ ---
git status                                  # де я і що змінилось
git add main.py                             # один файл у stage
git add .                                    # усі зміни у stage
git commit -m "add main.py"                 # зафіксувати
git commit -am "fix typo"                   # add + commit (лише tracked)
git log --oneline                            # коротка історія
git log --oneline --graph --all              # історія з деревом гілок
git diff                                     # що ще не в stage
git diff --staged                            # що піде в коміт
git show <хеш>                              # деталі одного коміту

# --- СКАСУВАННЯ ---
git restore main.py                         # відкотити правки у файлі
git restore --staged main.py                 # прибрати зі stage
git commit --amend -m "нове повідомлення"    # переписати останній коміт
git rm main.py                              # видалити файл з репо
git rm --cached .env                         # прибрати з Git, лишити на диску
git mv old.py new.py                        # перейменувати

# --- ВІДДАЛЕНИЙ РЕПОЗИТОРІЙ ---
git remote add origin git@github.com:user/repo.git
git remote -v                               # список remote
git push -u origin main                     # перший пуш (запам'ятати зв'язок)
git push                                    # далі – просто так
git pull origin main                        # забрати зміни й влити
git fetch                                   # лише дізнатися про зміни

# --- SSH ---
ssh-keygen                                  # згенерувати пару ключів
cat ~/.ssh/id_rsa.pub                       # публічний ключ → у GitHub/GitLab
ssh -T git@github.com                       # перевірити з'єднання
ssh -T git@gitlab.com

# --- ГІЛКИ ---
git branch                                  # список, * – поточна
git branch feature-login                    # створити
git checkout feature-login                  # перейти
git checkout -b feature-login               # створити і перейти
git switch -c feature-login                 # те саме, сучасний синтаксис
git push origin feature-login               # відправити гілку

```

```

git merge feature-login          # злити в поточну гілку
git branch -d feature-login      # видалити локально
git push origin --delete feature-login  # видалити на сервері

# --- ТИМЧАСОВІ ЗМІНИ (STASH) ---
git stash                        # відкласти зміни
git stash list                   # переглянути відкладене
git stash pop                   # повернути останнє
git stash -u                     # разом з новими файлами

# --- КОНФЛІКТИ ---
# 1) відкрити файл, прибрати <<<<<< ===== >>>>>>
git add <file>                  # 2) конфлікт вирішено
git commit                      # 3) завершити злиття
git merge --abort                # або скасувати все злиття

```

Три зони — одним поглядом

```

edit file → git add → git commit → git push
Working   Staging   Repository   GitHub /
directory area      (.git)         GitLab

```

Порядок дій, коли не розумієш, що робити

1. `git status` — Git сам напише, у якому ти стані й що можна зробити.
2. `git log --oneline` — де історія зараз.
3. `git diff` — що саме змінилось.

Мінімальний `.gitignore` для Python

```

.env/
venv/
__pycache__/
*.pyc
.env
.idea/
.vscode/
.pytest_cache/

```